Object Oriented Programming: Python BootCamp

HIT-emlyon BSc Program
April-May 2024, Harbin | © Imène BIRIGUI | Saeed VARASTEH

Lecture 02 : Control Structures

This lecture content will be about the control structures in python.

Recap: The last session, you have learned that you can store values in variables, and that these variables can be of different types, depending on which type of value they should store.

One of those types is the Boolean data type. It can only be one of two values: True or False.

Booleans can also result from comparison operators like "`==`" or "`>`", for example, 41 > 42 would result in False. In addition, there are also logical operators that act on Booleans, like: and, or and not.

Conditionals: If-statements

In this section, you will learn about If-statements. These are very useful, because they allow you to execute parts of your code based on whether a condition is met. Here is a short example:

```
if condition :
    bit of code
else :
    bit of code
```

As long as the statement after the keyword "if" evaluates to True, the indented code block will run. If the statement is False, all the indented parts will just be skipped when running the program.

We have also already mentioned "else", which you can use right after an if-statement. The code block after the "else" keyword will run if the code within the if-statement did NOT run.

There is also "elif", which lets you check additional conditions before moving on to the "else" part of your statement.

```
if [condition1] :
    bit of code
elif [condition2] :
    bit of code
else :
    bit of code
```

In []:
```
price = 42

if price > 100:
    print("expensive")

elif price > 50:
    print("moderate")

else:
    print("cheap")
```

In []:
```
name = "John"
if len(name) < 3:
    print("your name is short")
```

Functions

Functions are the building blocks of almost every Python program. They're where the real action takes place.

You've already seen how to use several functions, including print() and len(). These are all **built-in functions** because they come built into the Python language itself. You can also create **user-defined functions** that perform specific tasks.

Functions break code into smaller chunks, and are great for tasks that a program uses repeatedly. Instead of writing the same code each time the program need to perform the task, just call the function.

In the past few chapters you used functions like print() and len() to display text and determine the length of a string.

In []:
```
type(len)
```

You must **call** the function to tell Python to actually execute it.

Use parentheses to call the function.

In []:
```
len()
```

In this example, Python raises a TypeError when len() is called because len() expects **an argument**.

An **argument** is a value that gets passed to the function as **input**. Some functions can be called with no arguments, and some can take as many arguments as you like. len() requires exactly one argument.

Finally, when a function is done executing, it **returns** a value as **output**.

1. The function is **called**, and any arguments are passed to the function as input.
2. The function **executes**, and some action is performed with the arguments.
3. The function **returns**, and the original function call is replaced with the return value.

In []:
```
num_letters = len("emlyon")
print(num_letters)
```

First, len() is called with the argument "emlyon". The length of the string "emlyon" is calculated, which is the number 6. Then len() returns the number 6 and replaces the function call with the value.

something like this: num_letters = 6

Then Python assigns the value 6 to num_letters and continues executing any remaining lines of code in the program, which is print(num_letters).

Write Your Own Functions

As you write longer and more complex programs, you may find that you need to use the same few lines of code repeatedly. Or maybe you need to calculate the same formula with different values several times in your code.

In this section, you'll learn how to define your own functions so that you can avoid repeating yourself when you need to reuse code.

Every function has two parts:

1. The **function signature** defines the name of the function and any inputs it expects.
2. The **function body** contains the code that runs every time the function is used.

An example:

In []:
```
def multiply(x, y): # Function signature

    product = x * y # Function body

    return product # Return statement
```

It might seem odd to make a function for something as simple as the * operator. In fact, multiply is not a function you would probably write in a real-world scenario. But it makes a great first example for understanding how functions are created!

When Python reads a line beginning with the **def** keyword, it creates a new function. The function is assigned to a variable with the same name as the function name.

Since function names become variables, they must follow the same rules for variable names that you learned before.

You call a user-defined function just like any other function. Type the function name followed by a list of arguments in between parentheses.

For instance, to call multiply() with the argument 2 and 4, just type:

In []:
```
multiply(2,4)
```

Understand Scope in Python

When you assign a value to a variable, you are giving that value a name. Names are unique. For example, you can't assign the same name to two different numbers.

In []:
```
x = 2
x
```

In []:
```
x = 3
x
```

When you assign 3 to x, you can no longer recall the value 2 with the name x which makes sense.

Now, try running the following script:

In []:
```
x = 2

def func():
    x = 3
    print('Inside `func`, x has the value ',x)

func()

print('Outside `func`, x has the value ',x)
```

How does x still have the value 2 after calling func(), which changes the value of x to 3?

The answer is that the function func() has a **different scope** than the code that exists outside of the function.

The function body has what is known as a **local scope**, with its own set of names available to it. Code outside of the function body is in the **global scope**.

Scopes have a hierarchy. For example, consider the following:

In []:
```
x = 5

def func():
    y = x + 5

    print('Inside `func`, y has the value ',y)

func()

print('Outside `func`, x has the value ',x)
```

In []:
```
print('Outside `func`, y has the value ',y)
```

The variable y is in the local scope of func(). When Python executes the line y = x + 5, it looks for the variables x in the local scope. However, x DOESN'T exist there, so it moves up to the global scope and looks for x there.

EXERCISES 01 - PART TWO

Loops:

Since we are programmers we don't want to work hard but smart, so we need something to help us work smart and that's where loops come into play.

For example imagine your boss comes up to you and wants you to print out the numbers from 1 to 10. Well without knowing about loops you will probably write "print()" ten times with each number, but this is really slow and monotonous work, so we can introduce loops to take care of such chores for us!

Loops help us with tasks that sometimes are really similar, like the printing numbers example, or more complex, for example when we include conditionals or another for loop inside our existing for loop.

But don't worry, we will explain how all that work in the sections below!

For loops structure:

```
for [every_element] in [my_sequence]:
    do something
```

The structure might be confusing at first, but can be understood easily.

every_element is the variable that gets created using the for loop and that variable will iterate over the given sequence **my_sequence** from start to finish. So depending on how long that sequence is, the loop will "do something" for that many times.

As an example printing the numbers would be:

In []:
```
for i in range(1,11):
    print(i)
```

where **i** is the variable that iterates over our sequence, in this case a sequence of the numbers from 1 to 10 using the build-in function range().

range(1, 11) is the range of numbers 1, 2, 3, until 10. The first argument is the starting number, and the second argument is the endpoint, which is not included in the range.

br
co
an
pa

In []:
```
for i in range(1,11):
    if i == 5:
        continue
        print('we are here')

    print(i)
```

In []:
```
for i in range(1,11):
    if i == 5:
        break
        print('we are here')

    print(i)
```

In []:
```
for i in range(1,11):
    if i == 5:
        pass
        print('we are here')

    print(i)
```

While Loops structur

While Loop might be used for some more uncertain tasks where we only know when to stop, but not how many times a certain task needs to be done.

Again structure wise the While Loops can be understo like:

`while [condit`

`do whateve`

So before "starting the while loop, the **conditio** needs to be checked and only if it is True the while loop will actually "do whateve Imagine **conditio** as a conditior or just a boolean variable that would get set to False if a certain "event" would happen inside the while loop.

To use the same printing numbers example a while loop would look like this:

In []:
```
counter = 1
while counter < 11:
    print(counter)
    counter = counter + 1
```

So the cons is True as long as the **cour** is sma than 11 and once it is not the whil loop will stop

This is an exar whe bott loop diffe sinc the for loop just itera over a give seq and is finis in that sens and the whil loop actu coul run fore (if you are not care if the cons neve gets Fals

Ano diffe is the vari in use, sinc the for loop actu crea a vari to itera over the seq the whil loop does and that why we have to crea one for ours

In []:

EXERCISES 01 - PART THREE & FOUR